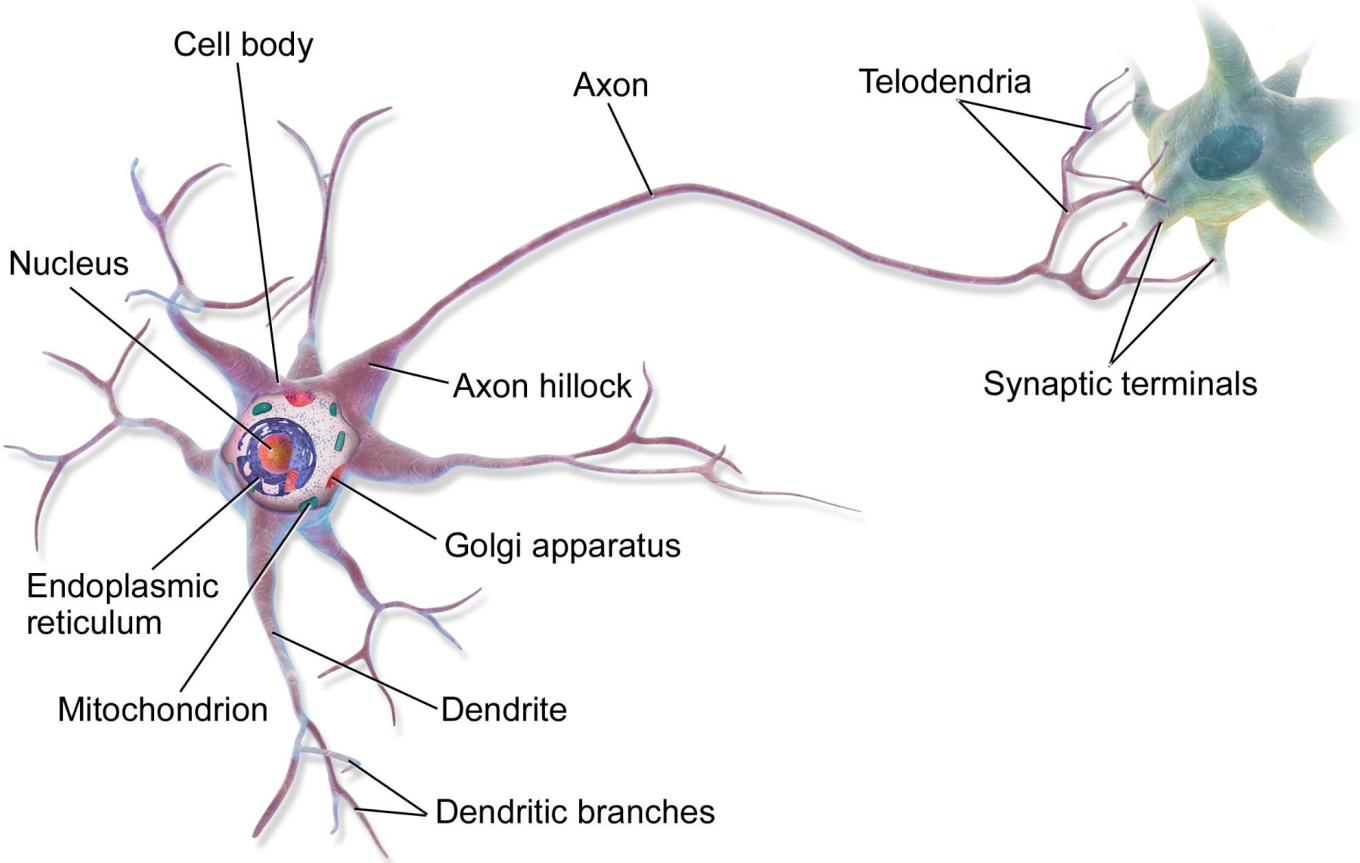
Neural Networks

NLP II 2026

Assoc. Prof. Attapol Thamrongrattanarit

Neural Networks

- A neural network is a machine learning (AI) model inspired by and based loosely on the human brain that processes information through interconnected nodes or neurons.


Cell body
Axon
Telodendria
Nucleus
Axon hillock
Synaptic terminals
Golgi apparatus
Endoplasmic
reticulum
Mitochondrion
Dendrite
Dendritic branches

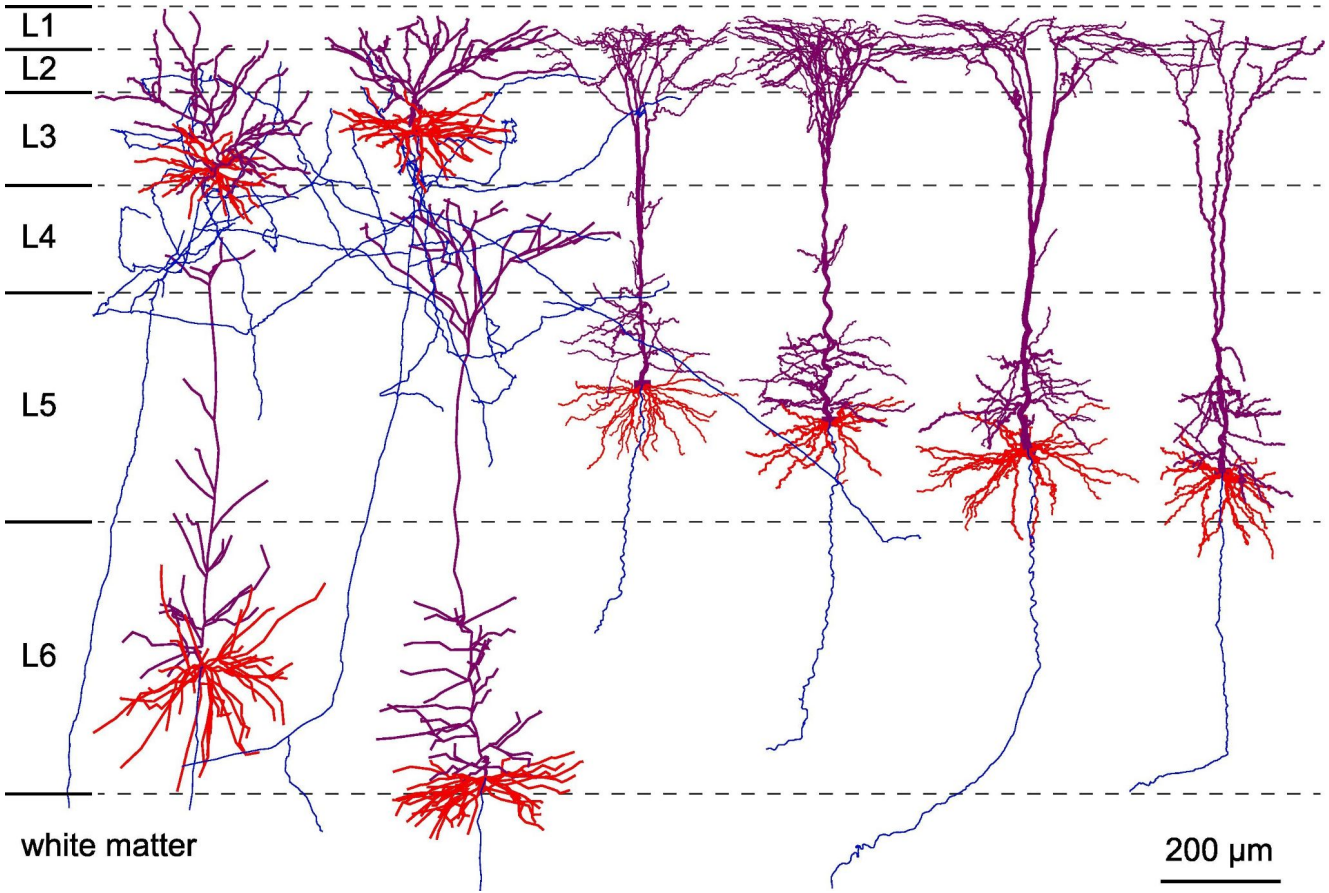
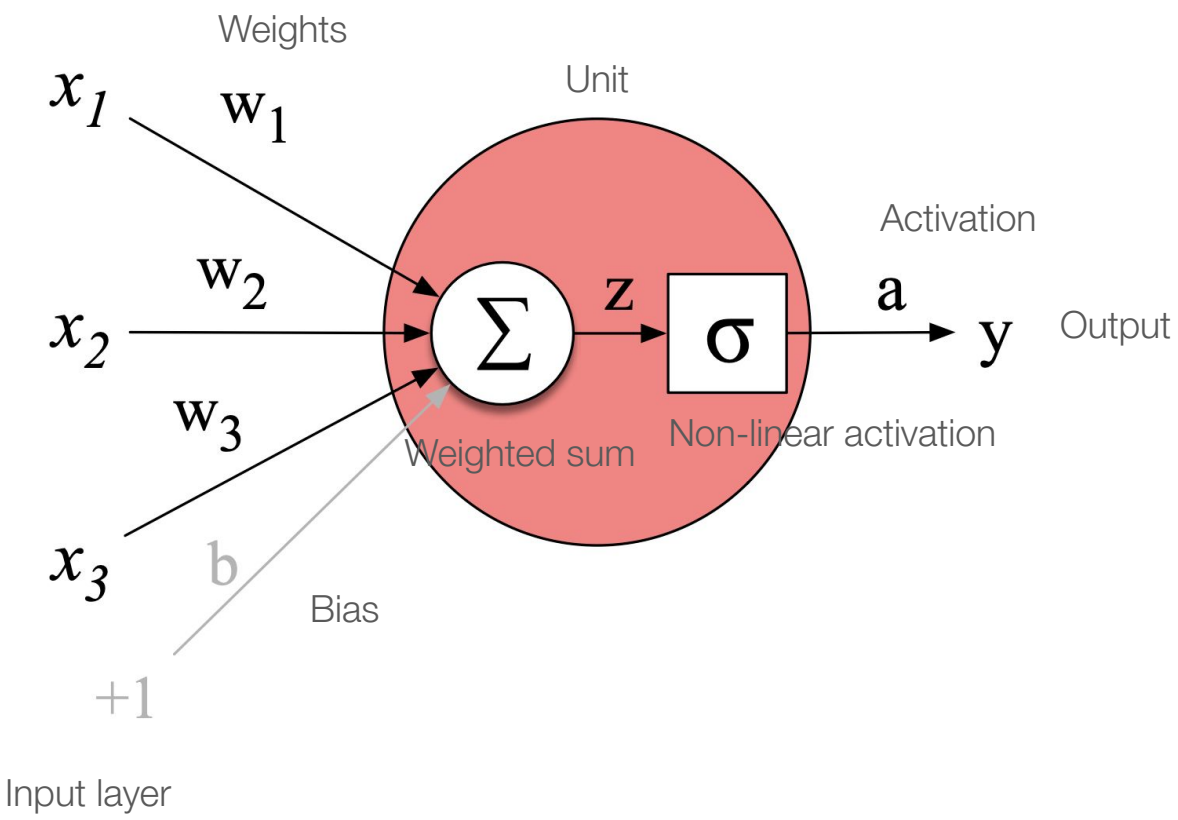


Image from Georgiev et al., 2020

Neural Network Unit



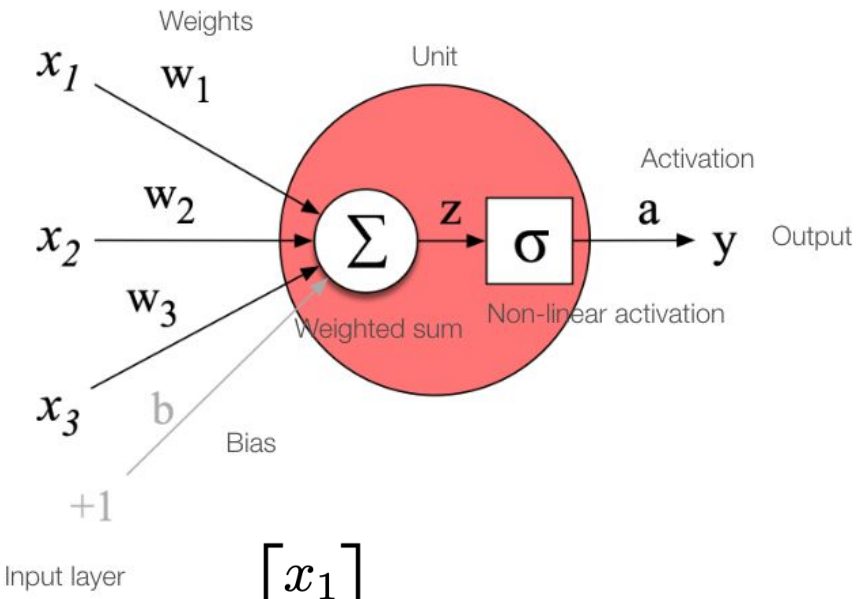
Neural Unit

$$z = \sum_{i=1}^{n} w_i x_i + b$$

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

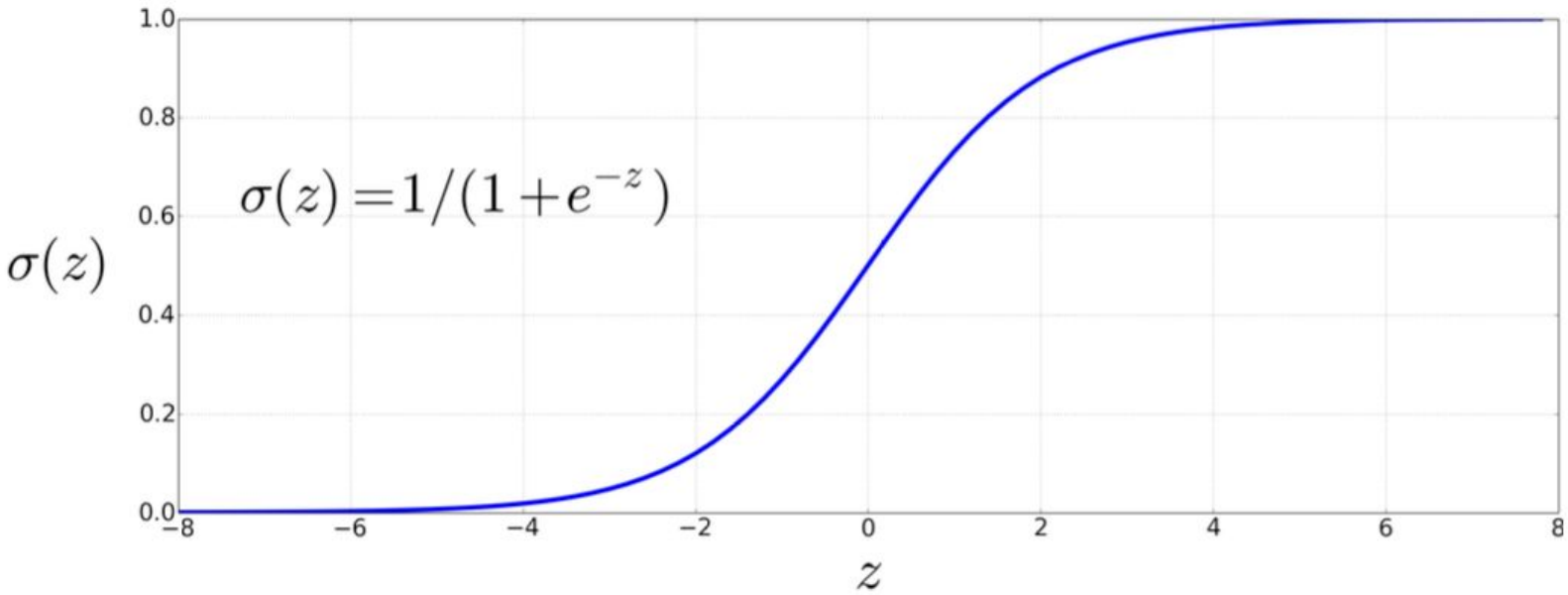
$$y = a = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$$



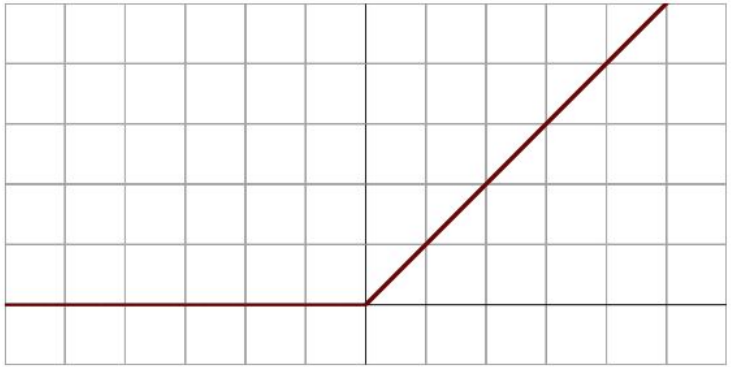
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Non-linear activation function: sigmoid



$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))}$$

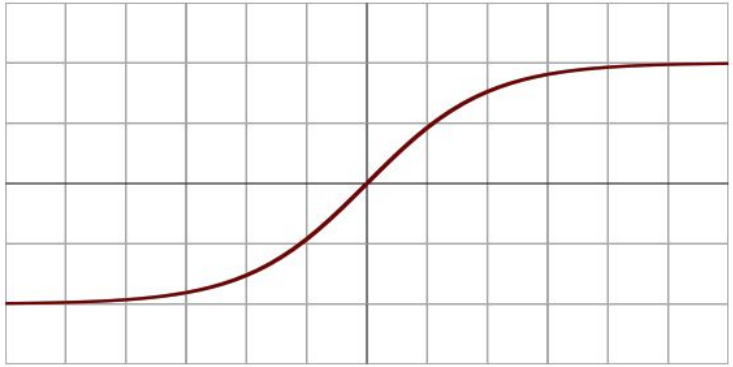
Non-linear activation function: ReLU and tanh



$$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$$

$$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$$

Rectified Linear Unit (ReLU) activation function



$$f(x) = \tanh(x) = \frac{(e^{x} - e^{-x})}{(e^{x} + e^{-x})}$$

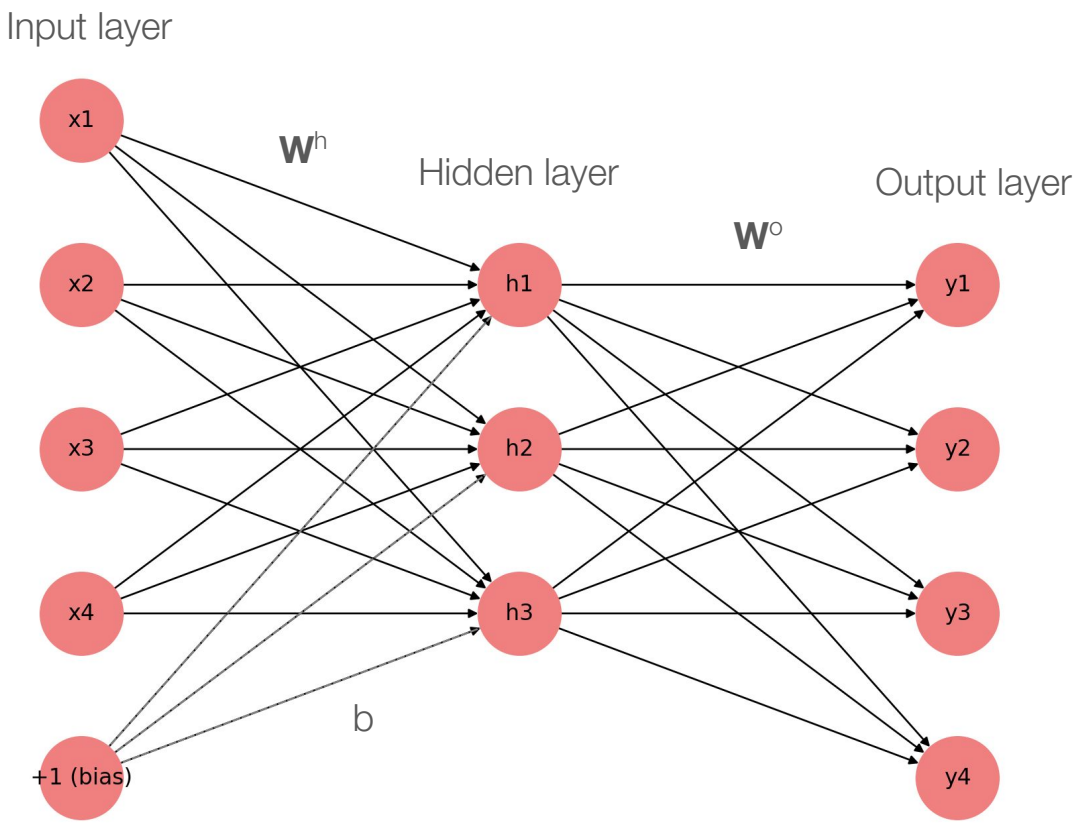
$$f'(x) = 1 - f(x)^2$$

Hyperbolic tangent activation function

Feedforward Neural Networks

Hidden layer and hidden unit

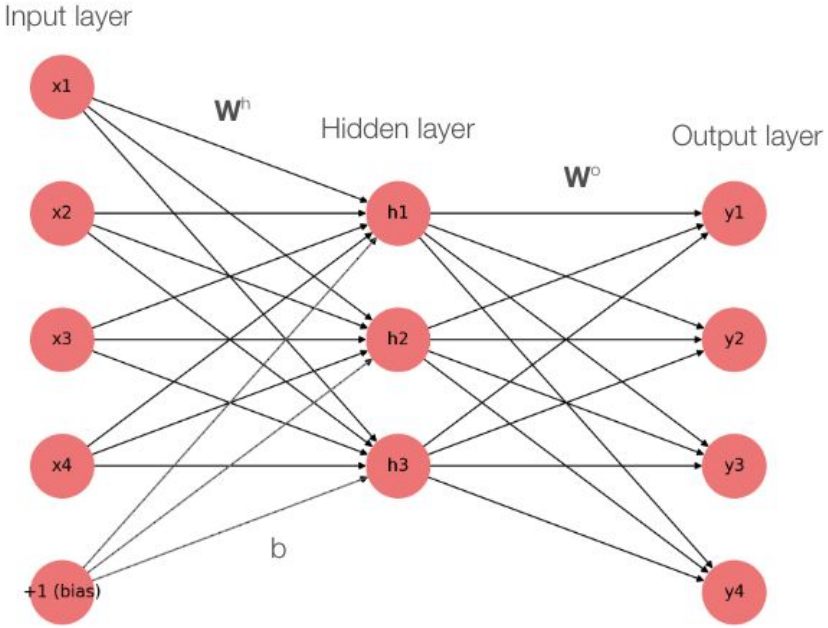
- A hidden unit is a neural unit (taking a weighted sum of its inputs and then applying non-linear activation function)
- In a standard setting, the input and the hidden layers are fully-connected, which means each hidden unit in the hidden layer sums over all the input units.



A two-layer feedforward neural network (one hidden layer and one output layer)

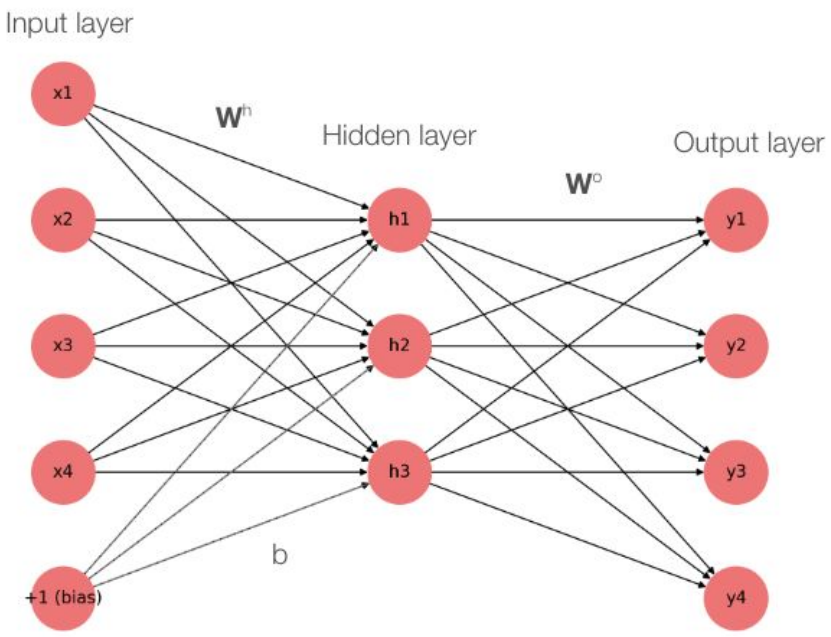
$$\mathbf{z} = W^h\mathbf{x} + b = \begin{bmatrix} 0.3 & -0.7 & 0.5 & 0.1 \\ -0.2 & 0.8 & -0.4 & 0.3 \\ 0.6 & -0.5 & 0.2 & -0.3 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.2 \\ 0.1 \\ 0.9 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.3 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.25 \\ -0.01 \\ 0.15 \end{bmatrix}$$

$$\mathbf{h} = \sigma(\mathbf{z}) = \begin{bmatrix} \sigma(0.25) \\ \sigma(-0.01) \\ \sigma(0.15) \end{bmatrix} = \begin{bmatrix} 0.5622 \\ 0.4975 \\ 0.5374 \end{bmatrix}$$



$$\mathbf{h} = \sigma(\mathbf{z}) = \begin{bmatrix} \sigma(0.25) \\ \sigma(-0.01) \\ \sigma(0.15) \end{bmatrix} = \begin{bmatrix} 0.5622 \\ 0.4975 \\ 0.5374 \end{bmatrix}$$

$$W^o h = \begin{bmatrix} 0.2 & -0.3 & 0.5 \\ -0.1 & 0.4 & -0.2 \\ 0.3 & -0.6 & 0.1 \\ 0.5 & 0.2 & -0.4 \end{bmatrix} \begin{bmatrix} 0.5622 \\ 0.4975 \\ 0.5374 \end{bmatrix} = \begin{bmatrix} 0.2319 \\ 0.0353 \\ -0.0761 \\ 0.1656 \end{bmatrix}$$

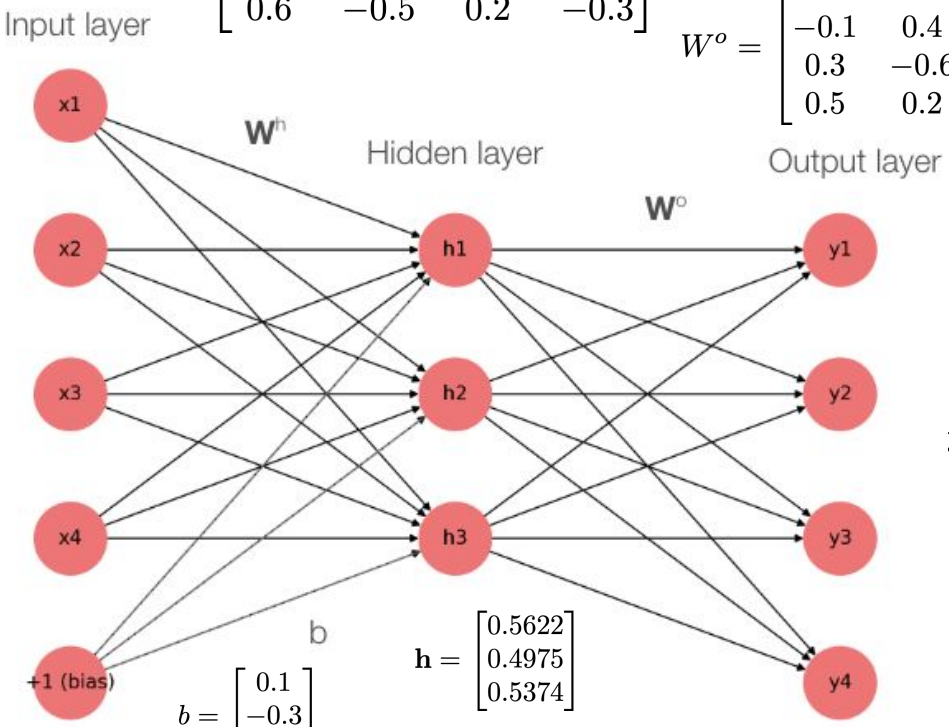


$$\mathbf{y} = \text{softmax}(W^o h) = \frac{e^{W^o h}}{\sum e^{W^o h}} = \begin{bmatrix} \frac{e^{0.2319}}{\sum e^{W^o h}} \\ \frac{e^{0.0353}}{\sum e^{W^o h}} \\ \frac{e^{-0.0761}}{\sum e^{W^o h}} \\ \frac{e^{0.1656}}{\sum e^{W^o h}} \end{bmatrix} = \begin{bmatrix} 0.2863 \\ 0.2352 \\ 0.2104 \\ 0.2680 \end{bmatrix}$$

$$W^h = \begin{bmatrix} 0.3 & -0.7 & 0.5 & 0.1 \\ -0.2 & 0.8 & -0.4 & 0.3 \\ 0.6 & -0.5 & 0.2 & -0.3 \end{bmatrix}$$

$$W^o = \begin{bmatrix} 0.2 & -0.3 & 0.5 \\ -0.1 & 0.4 & -0.2 \\ 0.3 & -0.6 & 0.1 \\ 0.5 & 0.2 & -0.4 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 0.5 \\ 0.2 \\ 0.1 \\ 0.9 \end{bmatrix}$$



$$b = \begin{bmatrix} 0.1 \\ -0.3 \\ 0.2 \end{bmatrix}$$

$$\mathbf{h} = \begin{bmatrix} 0.5622 \\ 0.4975 \\ 0.5374 \end{bmatrix}$$

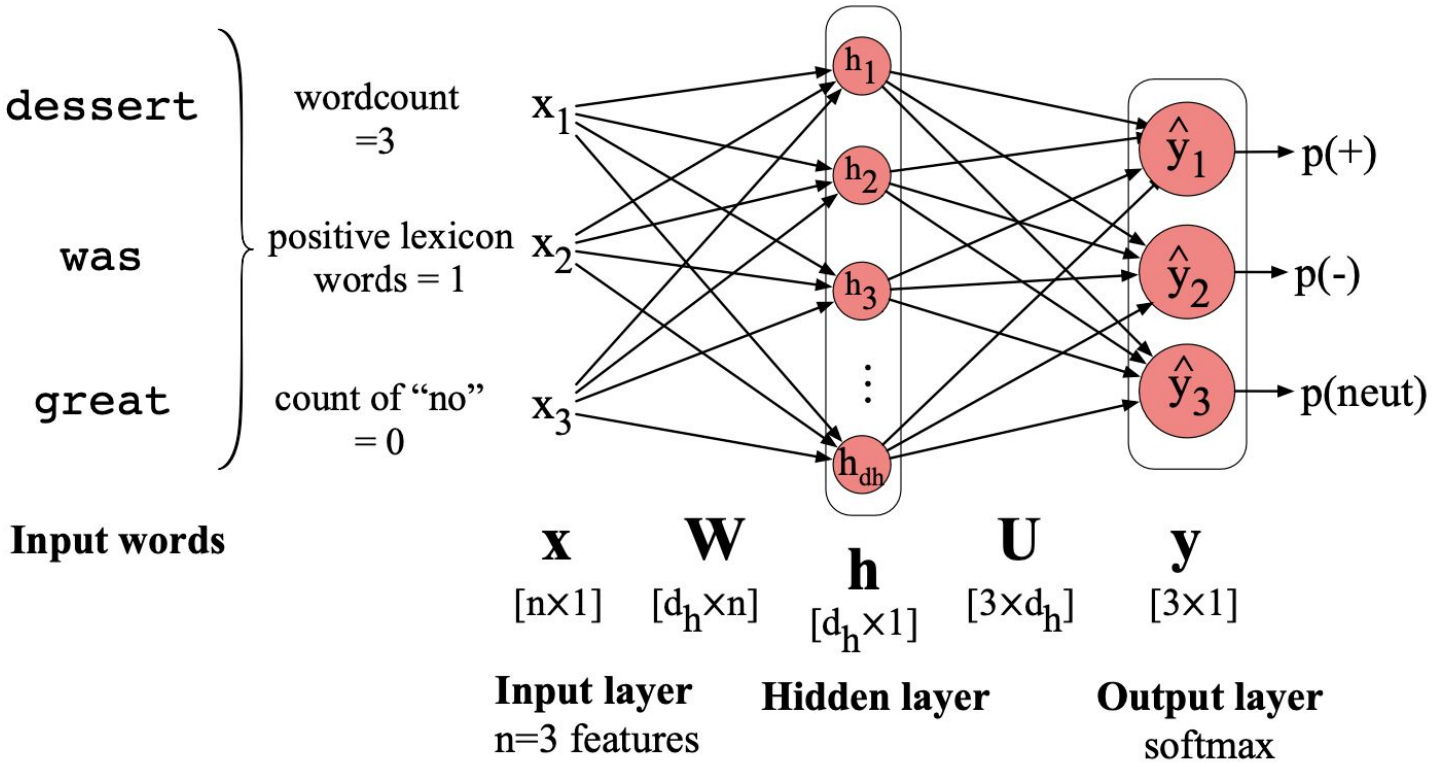
$$\mathbf{y} = \begin{bmatrix} 0.2863 \\ 0.2352 \\ 0.2104 \\ 0.2680 \end{bmatrix}$$

Feedforward neural network

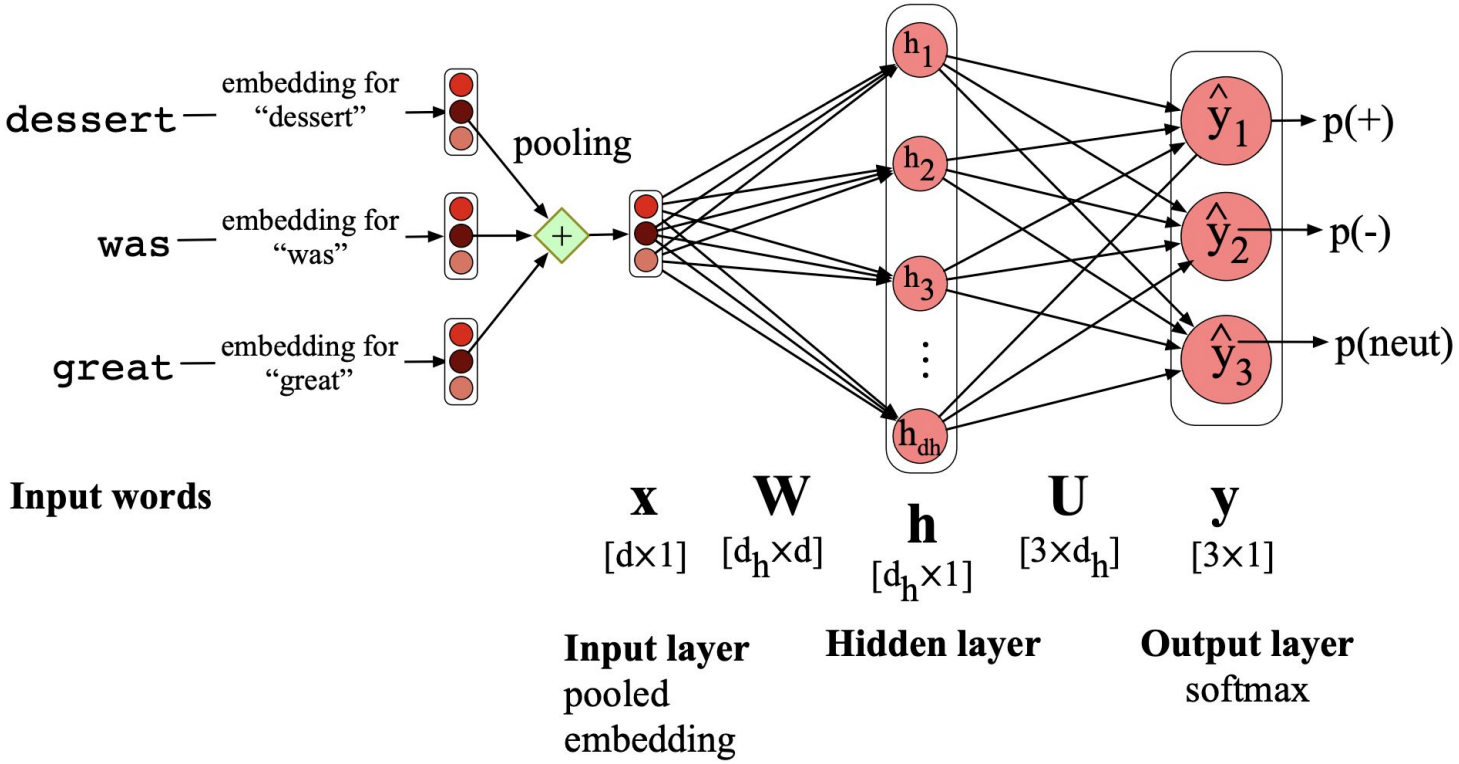
$$\mathbf{h} = \sigma(W^h \cdot \mathbf{x} + \mathbf{b})$$

$$\mathbf{y} = \text{softmax}(W^o \cdot \mathbf{h})$$

Feedforward Neural Networks for NLP: Classification



Feedforward network sentiment analysis with hand-built features



Feedforward network sentiment analysis with pooled word embeddings

Notation

- Suppose
  - X is n x d matrix (n = number of instances, d = embedding size)
  - W is d x k matrix (k = number of hidden units)
    - w_{ij} = the weight between x_i and h_j
- Then H = sigma(XW + **b**)
  - then h_{nj} = the hidden unit j for the instance n
- This is a more common notation for neural network

Training Neural Network

Training a neural network: loss function

- Training a neural network = finding the weights (parameters) that optimize (minimize) loss function.
- Define a loss function that measures the distance between the system output and the gold-standard output. We usually use cross-entropy loss:

$$L_{CE}(\hat{\mathbf{y}}, \mathbf{y}) = -\log \frac{\exp(\mathbf{z}_c)}{\sum_{j=1}^{K} \exp(\mathbf{z}_j)} \quad \text{(where } c \text{ is the correct class)}$$

Computing the gradient of a loss function

$$
\begin{aligned}
\frac{\partial L_{\mathrm{CE}}(\hat{\mathbf{y}}, \mathbf{y})}{\partial \mathbf{w}_{k,i}} &= -(\mathbf{y}_k - \hat{\mathbf{y}}_k)\mathbf{x}_i \\
&= -(\mathbf{y}_k - p(\mathbf{y}_k = 1|\mathbf{x}))\mathbf{x}_i \\
&= -\left(\mathbf{y}_k - \frac{\exp(\mathbf{w_k} \cdot \mathbf{x} + b_k)}{\sum_{j=1}^{K} \exp(\mathbf{w_j} \cdot \mathbf{x} + b_j)}\right)\mathbf{x}_i
\end{aligned}
$$

Reminder: Gradient Descent

- Update equation:

$$\theta_j^{new} = \theta_j^{old} - \underline{\alpha} \frac{\partial}{\partial \theta_j^{old}} J(\theta)$$

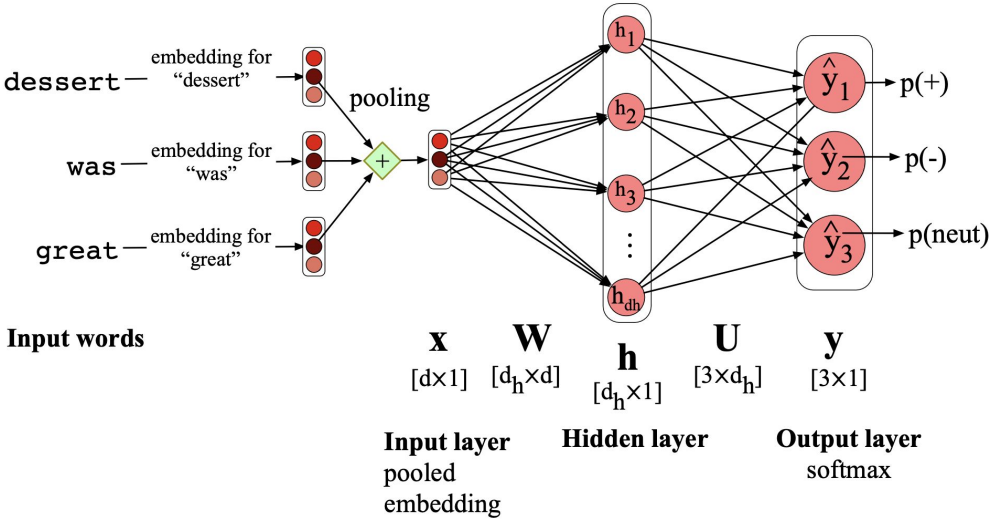
α = step size or learning rate

- Algorithm:

```
while True:
    theta_grad = evaluate_gradient(J,corpus,theta)
    theta = theta - alpha * theta_grad
```

Computing gradient

- When training neural networks, we must take the derivative with respect to all of the parameters in order to optimize loss.
- How do we compute derivative for all weights?



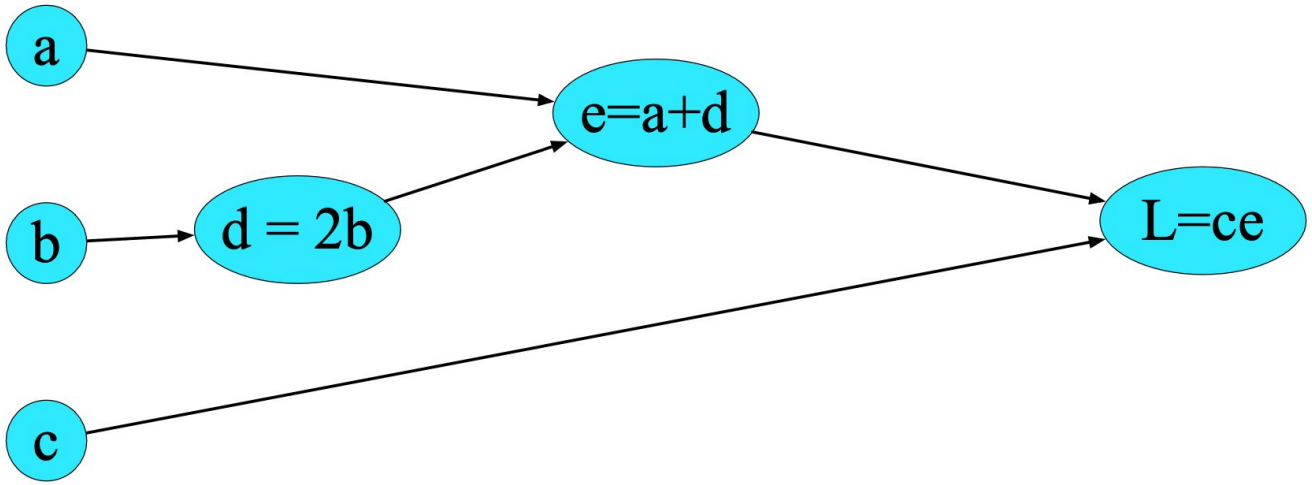
Automatic differentiation

- In practice, we use python library to compute derivative. But here's what you need to know
- When we set up a neural network, we define a computation graph, which represents what needs to be computed in the model.
- Computing derivatives can be done efficiently through this computation graph.

Example: $L(a,b,c) = c(a+2b)$

Computations:

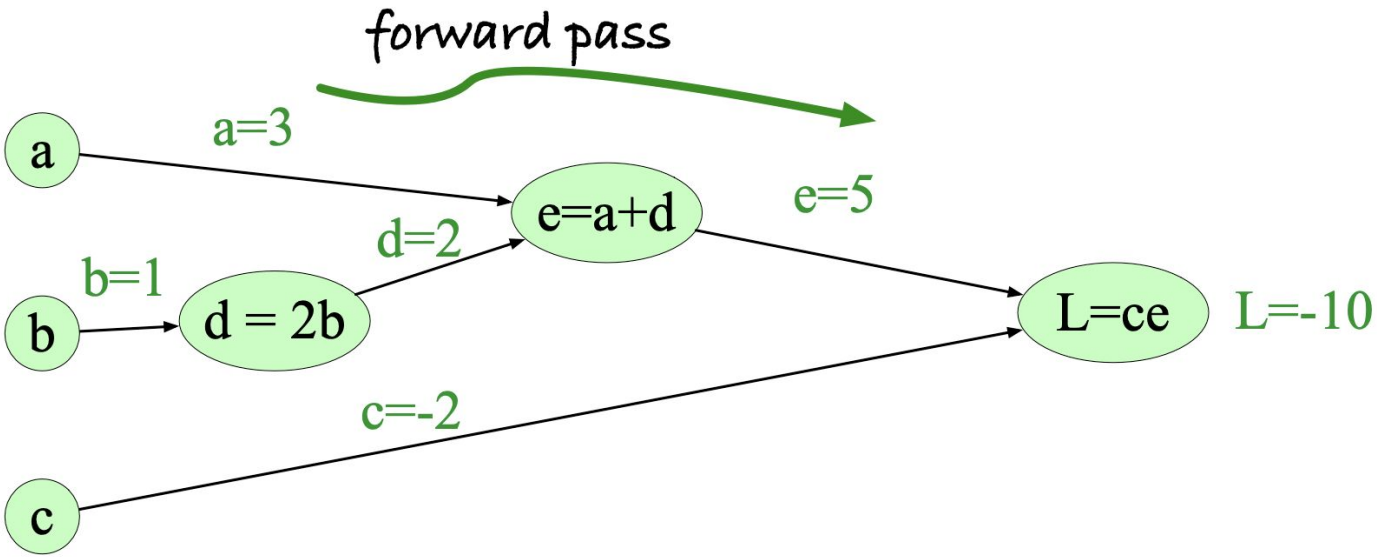
$$
\begin{aligned}
d &= 2*b \\
e &= a+d \\
L &= c*e
\end{aligned}
$$



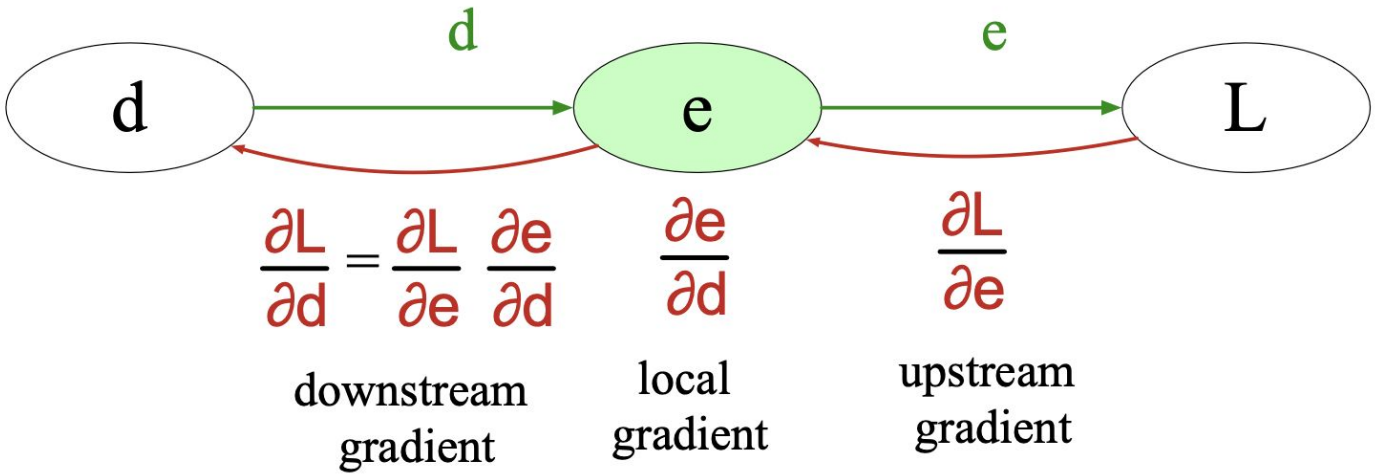
Example: $L(a,b,c) = c(a+2b)$

Computations:

$$
\begin{aligned}
d &= 2*b \\
e &= a+d \\
L &= c*e
\end{aligned}
$$



Passing the gradient back from the final node



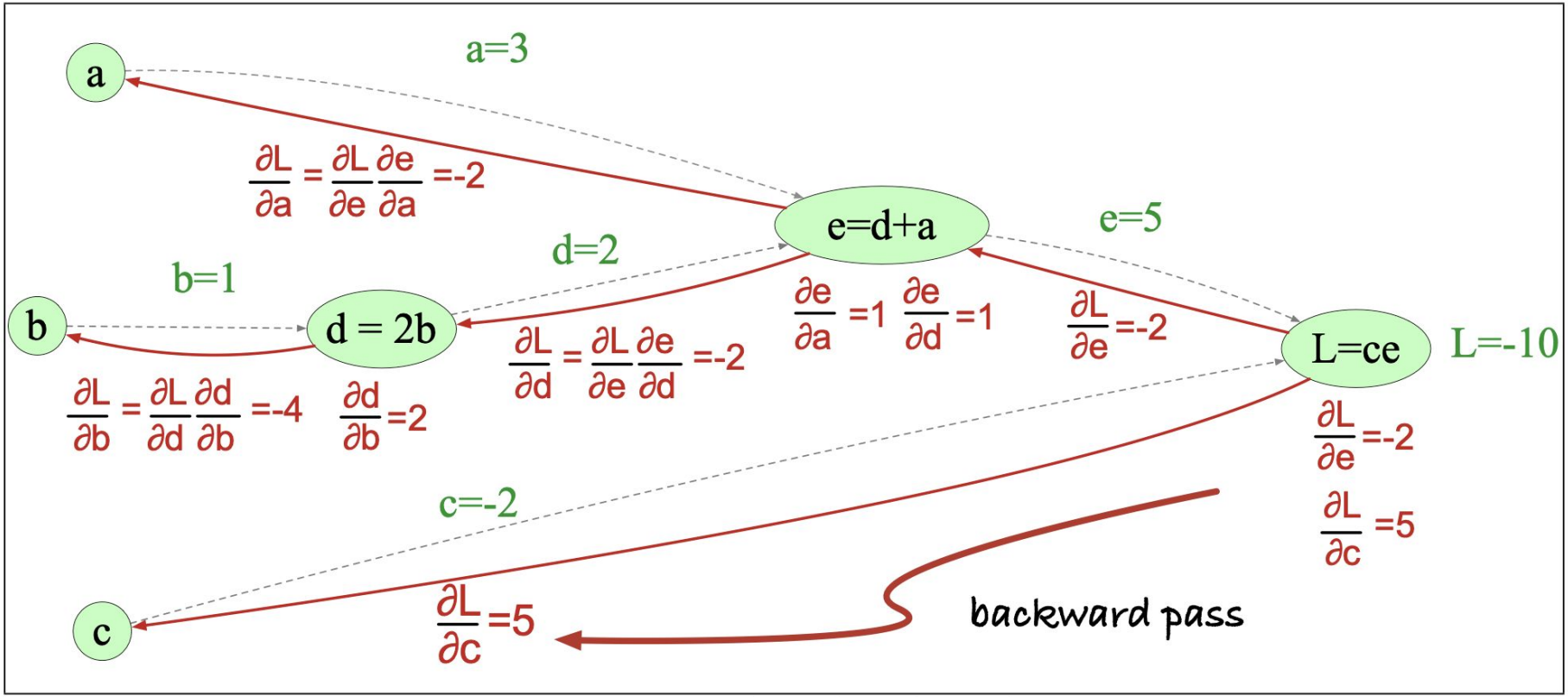
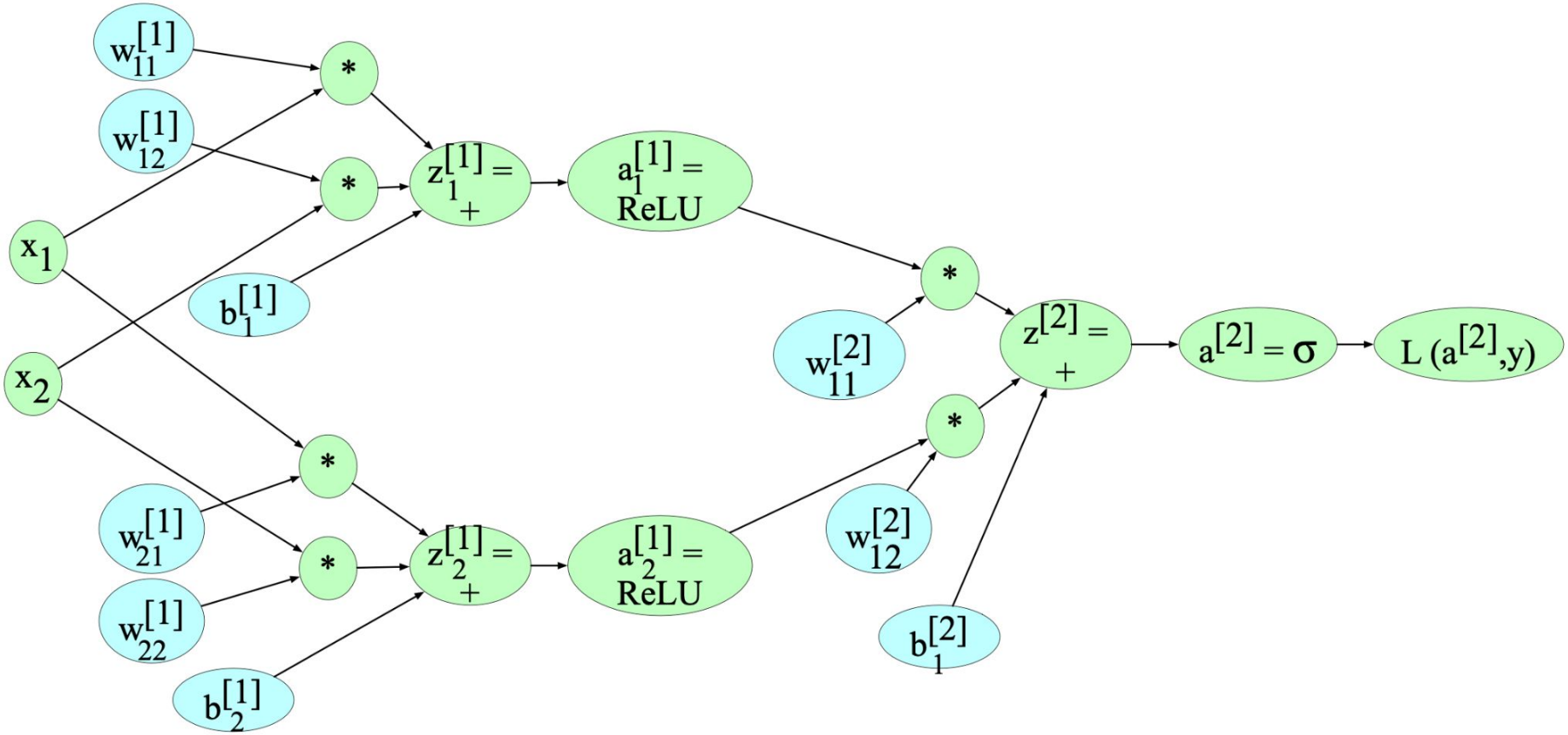


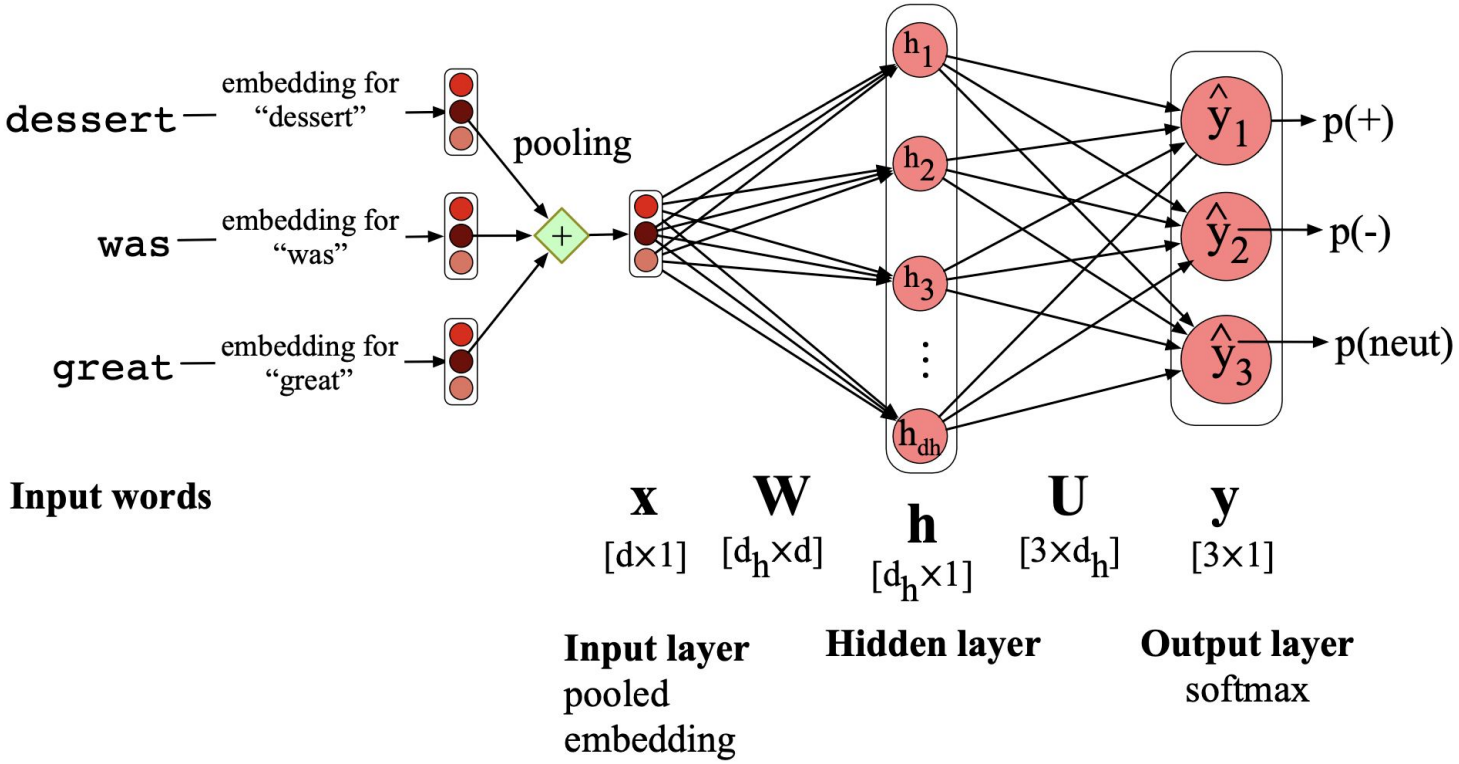
Figure 7.14 Computation graph for the function $L(a,b,c) = c(a+2b)$, showing the backward pass computation of $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$.


$w_{11}^{[1]}$
$w_{12}^{[1]}$
x_1
x_2
$b_1^{[1]}$
$w_{21}^{[1]}$
$w_{22}^{[1]}$
$b_2^{[1]}$
*
$z_1^{[1]} = +$
$z_2^{[1]} = +$
$a_1^{[1]} = \mathrm{ReLU}$
$a_2^{[1]} = \mathrm{ReLU}$
$w_{11}^{[2]}$
$w_{12}^{[2]}$
$b_1^{[2]}$
$z^{[2]} = +$
$a^{[2]} = \sigma$
$L(a^{[2]}, y)$

Automatic differentiation on computation graphs

- Set up the model in a computation graph in terms of the loss function
- Forward pass: compute the loss (we need it for computing gradient)
- Backward pass:
  - Compute local gradient from the last node in the graph
  - Pass the gradient backward in the graph
- In neural network training, we call this algorithm 'backpropagation'

Tricks for training neural networks



Feedforward network sentiment analysis with pooled word embeddings

Design choice

- Hyperparameters
  - Number of hidden layers
  - Number of hidden units
  - Embedding size
- How to choose? Evaluation on development/validation set

Three keys for training neural network models

- Gradient (learning signal) should be computed reasonably.
  ⇒ Use reasonable amount of training samples
- Update the parameters with the gradient scaled reasonably.
  ⇒ 0.1 x gradient too large? 0.01 x gradient too small?
- Decide when to stop learning

Stochastic gradient descent

```
theta = model parameters
for i in range(number of epochs):
    for x,y in training set:
        loss = compute_loss(theta, x, y)
        grad = compute_gradient(theta, x, y, loss)
        theta = theta - alpha * grad
```

Text	Label
Stochastic Gradient Descent	

Text	Label
Batch Gradient Descent	

Training vocab

- Iteration = One update to all parameters
- Epoch = One full training loop over the training set

Mini-batching

Computing gradients from all rows takes too much time per iteration

mini-batch size = 3
number of rows = 15
number of mini-batches = 5

1 epoch = 5 iterations

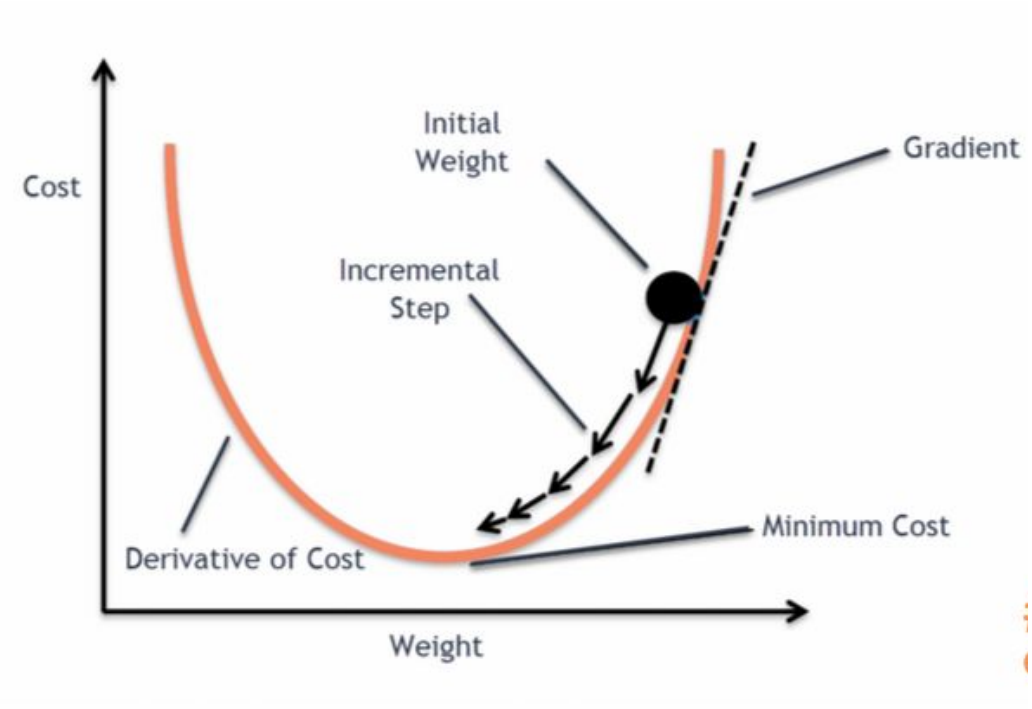
Text	Label

Three keys for training neural network models

- Gradient (learning signal) should be computed reasonably.
  ⇒ Use reasonable amount of training samples
- Update the parameters with the gradient scaled reasonably.
  ⇒ 0.1 x gradient too large? 0.01 x gradient too small?
- Decide when to stop learning

Gradient Descent

Compute errors from the gradient of loss functions to update the parameters to optimize loss function



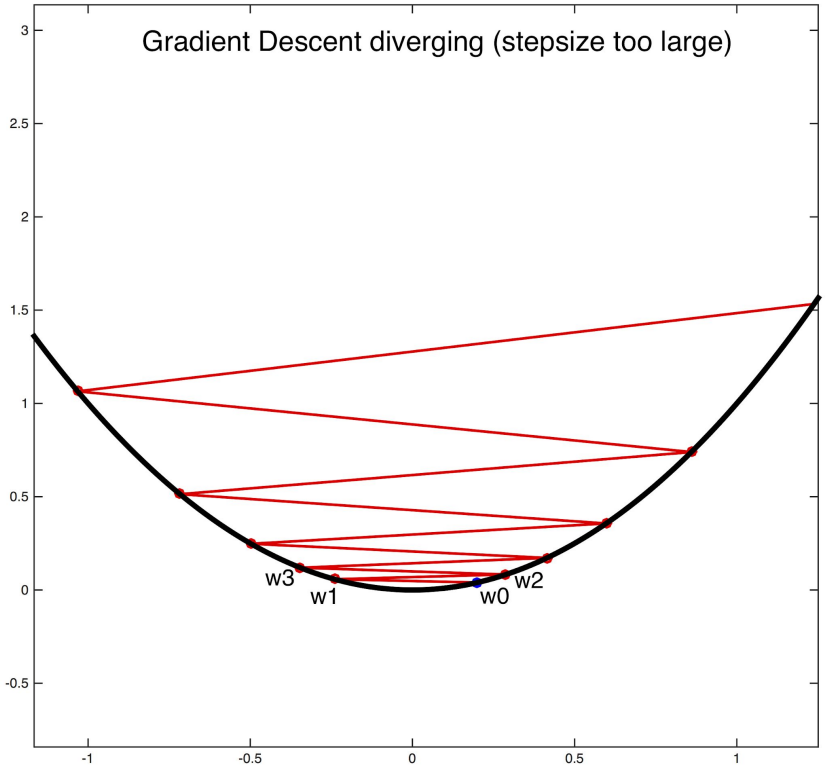
https://blog.clairvoyantsoft.com/the-ascent-of-gradient-descent-23356390836f

Problems

- Learning rate = step size = How quickly should we update?
  - If the step size is too small, we need more iterations. (each iteration costs you computation time)
  - If the step size is too large, we will never get good parameters because they oscillate too much.

Updating too quickly

Parameters oscillate over the optimal values.



Adaptive Learning Rate Optimizer

- Momentum
- RMSProp
- AdaGrad
- Adam

Momentum Intuition

Parameter	Recent Gradients	Mean	Momentum Step Size
θ_1	0.1, 0.2, 0.15	Small	Small step
θ_2	5, –4, 6	Small	Small step
θ_3	5, 4, 6	Large	Large step

Momentum

momentum

decay rate (usually = 0.9)

velocity

$$v_t = \beta v_{t-1} + (1 - \beta) g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

gradient at step t

parameter at step t

learning rate

Velocity is an exponentially decaying average of gradients

$$
\begin{aligned}
v_t &= \beta v_{t-1} + (1-\beta) g_t \\
&= \beta\big(\beta v_{t-2} + (1-\beta) g_{t-1}\big) + (1-\beta) g_t \\
&= \beta^2 v_{t-2} + \beta(1-\beta) g_{t-1} + (1-\beta) g_t
\end{aligned}
$$

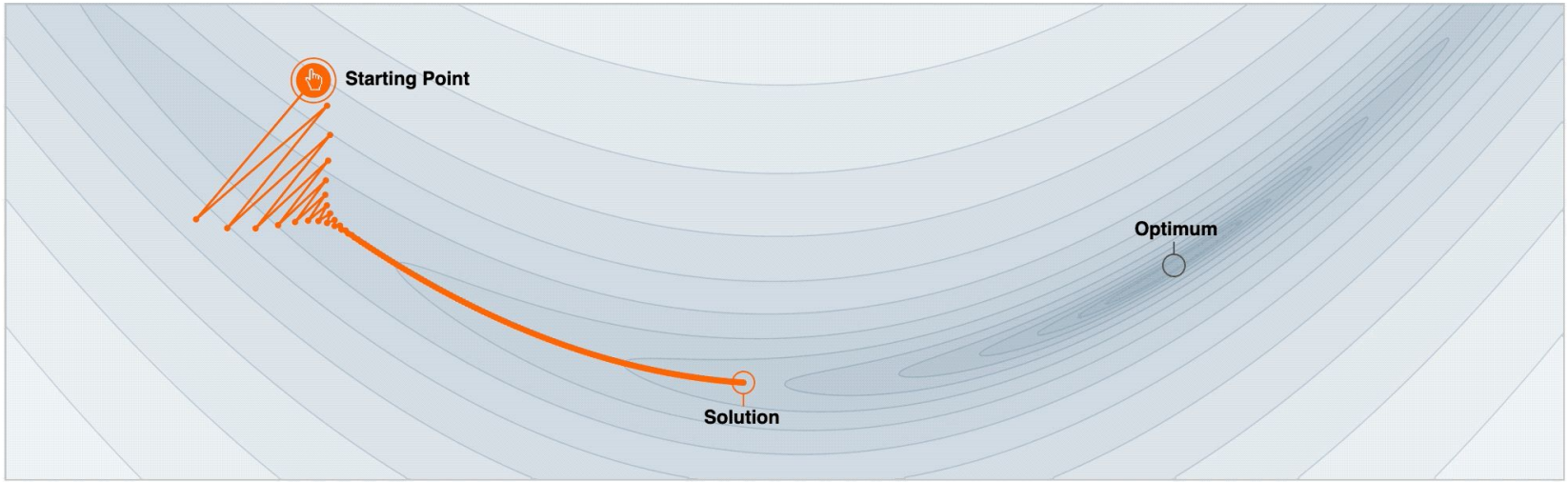
$$
v_t = (1-\beta) \sum_{k=0}^{t} \beta^k g_{t-k}
$$

https://distill.pub/2017/momentum/

Momentum

$$v_t = \beta v_{t-1} + (1-\beta) g_t$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$



Step-size α = 0.0030

0 0.003 0.006

Momentum β = 0.0

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

RMSProp Intuition

Parameter	Recent Gradients	Variance	RMSProp Step Size
θ_1	0.1, 0.2, 0.15	Small	Large step
θ_2	5, –4, 6	Large	Small step
θ_3	5, 4, 6	Small	Large step

RMSProp (Root Mean Squared)

decaying running average of squared gradient

decay rate

$$v_t = \beta v_{t-1} + (1-\beta) g_t^2$$

learning rate

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{v_t + \epsilon}} g_t$$

smoothing = small

Velocity is the exponentially decaying average of squared gradient

$$
\begin{aligned}
v_t &= \beta v_{t-1} + (1-\beta) g_t^2 \\
&= \beta \left(\beta v_{t-2} + (1-\beta) g_{t-1}^2 \right) + (1-\beta) g_t^2 \\
&= \beta^2 v_{t-2} + \beta (1-\beta) g_{t-1}^2 + (1-\beta) g_t^2 \\
&= (1-\beta) \sum_{k=0}^{t} \beta^k g_{t-k}^2
\end{aligned}
$$

What happens if beta = 0?

AdaGrad

learning rate

$$w_{t,i} = w_{t-1,i} - \frac{\eta}{\sqrt{\epsilon + \sum_{j=1}^{t-1} w_{j,i}'^2}} w'_{t,i}$$

ค่านี้จะใหญ่ขึ้นเรื่อยๆ ทำให้ก้าวช้าลงเรื่อย ๆ

Adam

- Adam Optimizer uses
  - mean gradients over the past few iterations
  - variance of gradients over the past few iterations

Adam

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_{t+1} = \theta_t - \alpha \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Adaptive Learning Rate

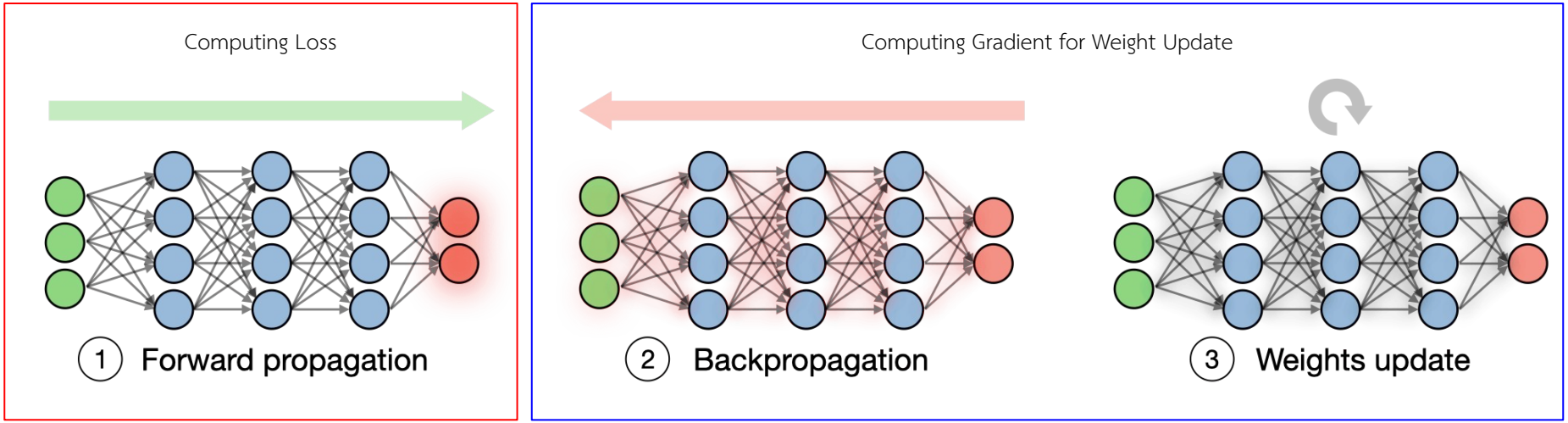
```
theta = model parameters
optimizer = AdaptiveOptimizer()  # e.g., Adam, RMSProp, etc.

for i in range(number of epochs):
    for x, y in minibatch_training set:
        loss = compute_loss(theta, x, y)
        grad = compute_gradient(theta, x, y, loss)
        optimizer.update_statistics(grad)
        alpha = optimizer.get_learning_rate()
        theta = theta - alpha * optimizer.modify_gradient(grad)
```

Optimizers

- There is no definite answer on which one is the best optimizer.
- Optimizers work based on
  - The decaying average of the gradients that we have seen so far (Momentum and Adam)
  - The decaying average of the squared gradients that we have seen so far (RMSProp and Adam)

Training Process



https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-deep-learning-tips-and-tricks

Adaptive Learning Rate + Minibatching

```
theta = model parameters
optimizer = AdaptiveOptimizer()  # e.g., Adam, RMSProp, etc.
data_loader = DataLoader(training set)

for i in range(number of epochs):
    for x_batch, y_batch in data_loader:
        loss = compute_loss(theta, x, y)
        grad = compute_gradient(theta, x, y, loss)
        optimizer.update_statistics(grad)
        alpha = optimizer.get_learning_rate()
        theta = theta - alpha * optimizer.modify_gradient(grad)
```

Three keys for training neural network models

- Gradient (learning signal) should be computed reasonably.
  ⇒ Use reasonable amount of training samples
- Update the parameters with the gradient scaled reasonably.
  ⇒ 0.1 x gradient too large? 0.01 x gradient too small?
- Decide when to stop learning

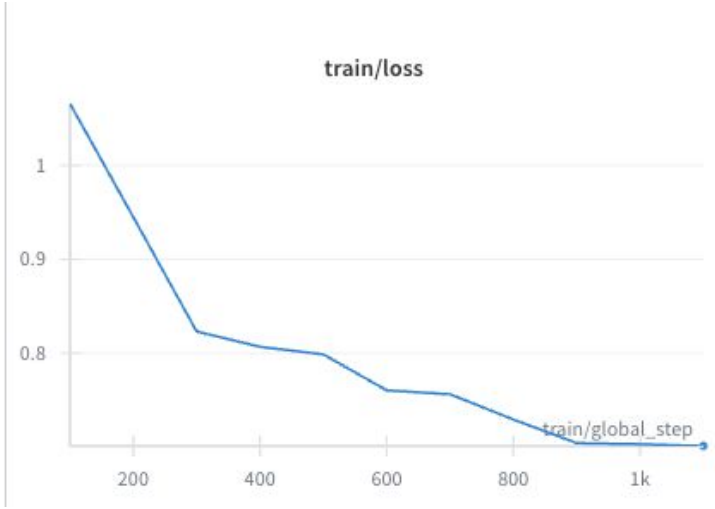
How do we know we have done enough?

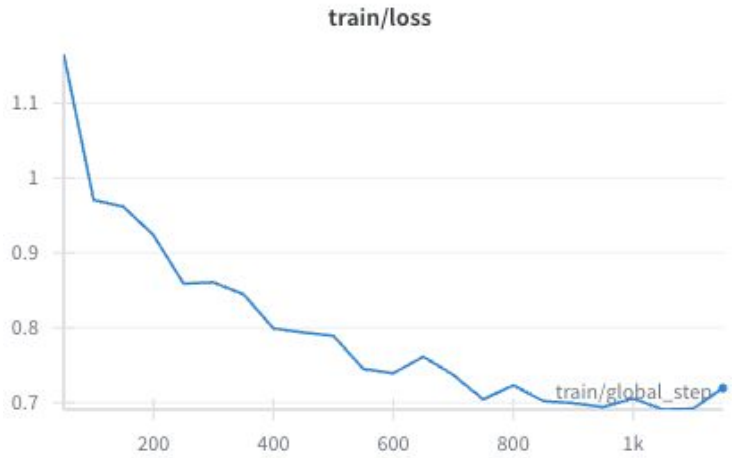
We stop updating when further updates don't meaningfully improve model performance. This is called early stopping. This decision is usually based on three signals:

- Training loss
- Validation loss
- Validation performance metrics

Training loss

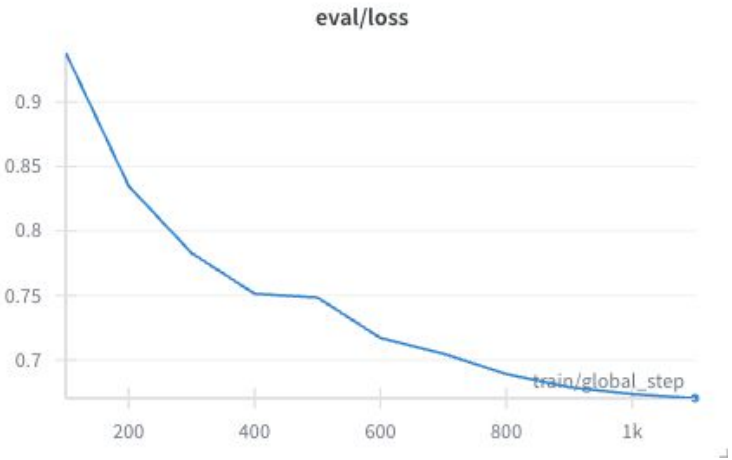
- Typically, the loss function drops quite rapidly at the beginning and slows down toward the end.
- If the loss graph looks 'flat enough,' then the model starts to overfit, so we know we have updated (more than) enough iterations.

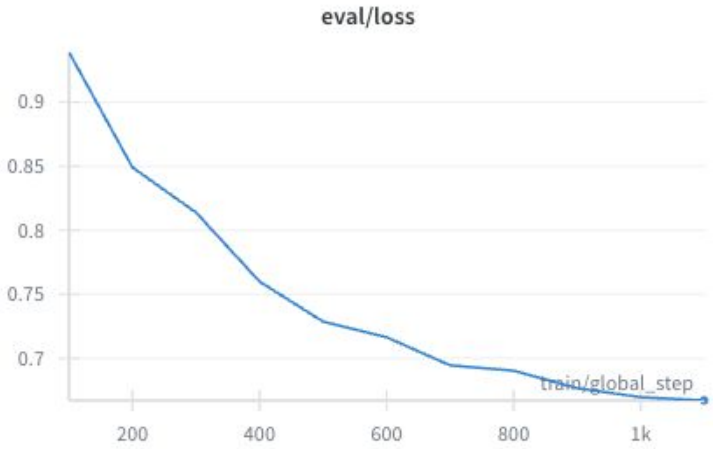




Validation loss

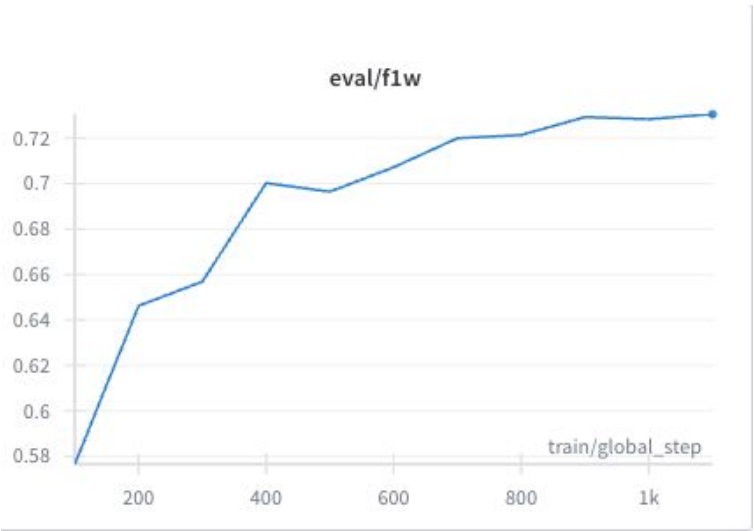
- The validation loss behaves the same way as the training loss.
- Optimizers are good at preventing overfitting, so more iterations are usually safer.

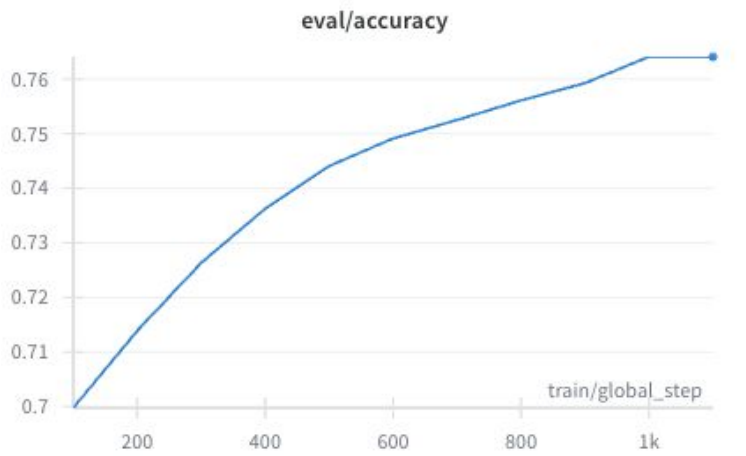




Validation performance

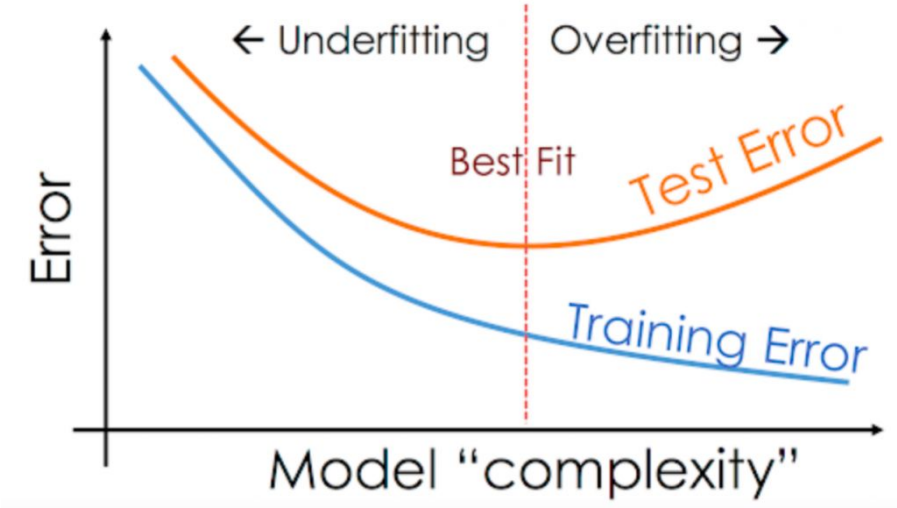
- Validation F1 (or other metrics) should rapidly increase and then plateau.
- Training F1 can reach 1.0. The model will not generalize well, so we need to validation F1 to help select the best model.





The Tension

Adding more parameters (more complexity) and/or training more iterations reduces the test error to a certain point and then increases it.



Early Stopping

- Overfitting refers to the situation where the model fits too closely to the training data, and as a result, it doesn't generalize to unseen data. This happens when we have a lot of features (e.g. bag-of-word features)
- Underfitting refers to the situation where the model is too simple to fit well to the training data, and as a result, it performs poorly on unseen data. This happens when we have too few features or the features are ineffective.
- In practice, early stopping means you monitor validation performance during training, remember the best checkpoint, and then restore that checkpoint at the end.

Conclusion: Neural Networks

- Neural networks are the most successful models in NLP because they can use word embeddings and are flexible enough to process any piece of text.
- Training neural networks to work well is still partly an art.
  - gradient turns to zero or loss turns NaN
  - batch size
  - training too long (overfitting) or training too little (underfitting)

Training Neural Networks

PyTorch Tutorial

- PyTorch is a very popular library for training neural networks for NLP.
- Preprocessing data is the most complicated step, which AI cannot really help you. You really have to control AI, and not the other way.

Data Preparation

1. Read data from raw text (word segment as necessary)
2. Create a vocabulary (Tokenizer)
3. Convert word strings into word indices
4. Pad sequences to be the same length

1. Read data from raw file

- A list of sentences (strings)
  - Each sentence should be tokens separated by space (in both English and Thai)

Example - word segmentation

อร่อยจังเลย
ไม่รู้อร่อยตรงไหน
พอกินได้
กินจนพุงกาง

อร่อย จัง เลย
ไม่ รู้ อร่อย ตรง ไหน
พอ กิน ได้
กิน จน พุง กาง

2. Create a vocabulary

- Tokenizer class in keras learns from the list of sentences
  - word_index
  - index_word
  - 0 is for padding
- The indices are sorted by frequencies.
  - If an index is greater than the vocab size, then it's converted to the same index for out-of-vocabulary index (= vocab size)

Example vocabulary - Vocab size = 10

index	word
0	<PADDING>
1	อร่อย
2	กิน
3	จัง
4	เลย
5	ไม่
6	รู้
7	ตรง
8	ไหน
9	พอ
10	ได้
11	จน
12	พุง
13	กาง

index	word
0	<PADDING>
1	อร่อย
2	กิน
3	จัง
4	เลย
5	ไม่
6	รู้
7	ตรง
8	ไหน
9	พอ
10	<UNK>

3. Convert strings to indices

- Tokenizer class uses the vocabulary (word_index and index_word) to convert each word string into its index
- We get a sequence of indices

Example

อร่อย จัง เลย
1 3 4

ไม่ รู้ อร่อย ตรง ไหน
5 6 1 7 8

พอ กิน ได้
9 2 10

กิน จน พุง กาง
2 10 10 10

index	word
0	<PADDING>
1	อร่อย
2	กิน
3	จัง
4	เลย
5	ไม่
6	รู้
7	ตรง
8	ไหน
9	พอ
10	<UNK>

4. Pad sequences to the same length

- Each list of indices must be of the same length.
  - Pad empty spots with 0
  - Truncate the sequence to the same length. Why is this bad?
- We will run the model on the whole list (sequence) regardless of the actual length of the sentence.
  - Choose the sequence length carefully. What if it's too long?

Example - Set the sequence length to 8

อร่อย จัง เลย
1 3 4
0 0 0 0 0 1 3 4

ไม่ รู้ อร่อย ตรง ไหน
5 6 1 7 8
0 0 0 5 6 1 7 8

พอ กิน ได้
9 2 10
0 0 0 0 0 9 2 10

กิน จน พุง กาง
2 10 10 10
0 0 0 0 2 10 10 10

index	word
0	<PADDING>
1	อร่อย
2	กิน
3	จัง
4	เลย
5	ไม่
6	รู้
7	ตรง
8	ไหน
9	พอ
10	<UNK>

Example - Set the sequence length to 8

อร่อย จัง เลย
1 3 4
0 0 0 0 0 1 3 4

ไม่ รู้ อร่อย ตรง ไหน
5 6 1 7 8
0 0 0 5 6 1 7 8

พอ กิน ได้
9 2 10
0 0 0 0 0 9 2 10

กิน จน พุง กาง
2 10 10 10
0 0 0 0 2 10 10 10

We get the training data matrix:

0 0 0 0 0 1 3 4
0 0 0 5 6 1 7 8
0 0 0 0 0 9 2 10
0 0 0 0 2 10 10 10